

SERVICIO NACIONAL DE APRENDIZAJE**SENA**

Centro de Gestión de Mercados, Logística y Tecnologías de la Información

Tecnólogo en Análisis y Desarrollo de Software

**INFORME TÉCNICO DE PLAN DE TRABAJO
HERRAMIENTAS Y TECNOLOGÍAS DE VERSIONAMIENTO
PARA LA CONSTRUCCIÓN DE SOFTWARE**

Evidencia: GA7-220501096-AA1-EV01

Proyecto: Trueque Match

Aplicación móvil y web de intercambio solidario sin dinero

Estudiante: Gerson Jonnathan López Oviedo

Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Bogotá D.C., Mayo de 2026

TABLA

DE CONTENIDO	3
1. Introducción	4
2. Objetivo.....	5
2.1 Objetivo General	5
2.2 Objetivos Específicos.....	5
3. Control de Versiones	6
3.1 Definición y Conceptos Fundamentales	6
3.2 Tipos de Sistemas de Control de Versiones	7
3.2.1 Sistemas Locales.....	7
3.2.2 Sistemas Centralizados (CVCS).....	8
3.2.3 Sistemas Distribuidos (DVCS).....	8
3.2.4 Sistemas Basados en Bloqueos (Pessimistic Locking)	9
3.2.5 Sistemas Basados en Fusión Optimista (Optimistic Merging).....	10
4. Ventajas de las Herramientas de Versionamiento	12
4.1 Calidad del Software.....	12
4.2 Visibilidad y Trazabilidad	12
4.3 Aceleración del Desarrollo	13
4.4 Colaboración Efectiva.....	14
4.5 Respaldo y Recuperación ante Desastres	14
4.6 Documentación Implícita del Proyecto.....	15
5. Principales Herramientas de Control de Versiones.....	16
5.1 Git	16
5.2 GitHub	16
5.3 GitLab.....	17
5.4 Bitbucket	17
5.5 Apache Subversion (SVN).....	18
5.6 Azure DevOps (TFVC y Azure Repos)	18
6. Cómo las Herramientas de Versionamiento Optimizan el Control de Versiones.....	20
6.1 Estrategias de Ramificación (Branching Strategies).....	20
6.2 Integración Continua y Despliegue Continuo (CI/CD)	21
6.3 Revisión de Código Estructurada	21
6.4 Etiquetado Semántico de Versiones	22
6.5 Resolución Asistida de Conflictos	22

7. Selección de Herramientas para Trueque Match	23
7.1 Características del Proyecto	23
7.2 Criterios de Selección	23
7.3 Herramienta Seleccionada: Git + GitHub	24
7.3.1 Justificación de la Selección de Git	24
7.3.2 Justificación de la Selección de GitHub	24
7.3.3 Estrategia de Versionamiento para Trueque Match	25
8. Conclusiones	27
9. Referencias	28

DE CONTENIDO

1. Introducción

En la actualidad, el desarrollo de software es una disciplina que exige no solo habilidades técnicas en programación, sino también la capacidad de gestionar de manera ordenada y trazable los cambios que se producen en el código fuente a lo largo del tiempo. A medida que los proyectos crecen en tamaño y complejidad, y que los equipos de trabajo se amplían o se distribuyen geográficamente, la necesidad de contar con mecanismos robustos de control de versiones se vuelve indispensable para garantizar la integridad, calidad y continuidad del desarrollo.

El control de versiones es una práctica fundamental en la ingeniería de software que permite registrar cada cambio realizado en el código fuente, identificar quién realizó ese cambio, cuándo lo hizo y por qué razón. Más allá de ser una simple herramienta de respaldo, un sistema de control de versiones moderno habilita el trabajo paralelo en múltiples funcionalidades, facilita la resolución de conflictos entre cambios concurrentes y proporciona un historial completo que puede ser auditado en cualquier momento.

El presente documento corresponde a la evidencia GA7-220501096-AA1-EV01 del programa Tecnólogo en Análisis y Desarrollo de Software del SENA. Tiene como propósito investigar en profundidad el concepto de control de versiones, los tipos de sistemas existentes, sus ventajas, las herramientas más utilizadas en la industria y la manera en que estas optimizan el proceso de desarrollo. Finalmente, se realiza una selección justificada de las herramientas más adecuadas para el proyecto Trueque Match, una aplicación móvil y web orientada a facilitar el intercambio de bienes, servicios, conocimientos y experiencias entre personas, sin necesidad de dinero.

2. Objetivo

2.1 Objetivo General

Elaborar un informe técnico que investigue, analice y seleccione las herramientas y tecnologías de control de versiones más adecuadas para la construcción del software Trueque Match, con base en los fundamentos teóricos del versionamiento y las características específicas del proyecto.

2.2 Objetivos Específicos

- Definir el concepto de control de versiones y explicar su importancia en el desarrollo de software moderno.
- Identificar y caracterizar los principales tipos de sistemas de control de versiones: centralizados, distribuidos, basados en bloqueos y basados en fusión optimista.
- Analizar las ventajas que ofrecen las herramientas de versionamiento en aspectos como calidad, visibilidad, aceleración y colaboración.
- Describir las principales herramientas de control de versiones disponibles en el mercado y su adopción en la industria.
- Explicar cómo las herramientas de versionamiento optimizan el proceso de desarrollo de software.
- Seleccionar y justificar la herramienta de versionamiento más apropiada para el proyecto Trueque Match.

3. Control de Versiones

3.1 Definición y Conceptos Fundamentales

El control de versiones, también denominado gestión de configuración de software (SCM, por sus siglas en inglés), es el proceso de registrar y administrar los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, de manera que sea posible recuperar versiones específicas en cualquier momento futuro. Chacon y Straub (2014) definen un sistema de control de versiones como una herramienta que permite a los equipos de desarrollo colaborar de manera eficiente, manteniendo un historial completo de todos los cambios realizados sobre el código fuente.

Para comprender este concepto de manera intuitiva, puede pensarse en el control de versiones como el historial de cambios de un documento de texto compartido, pero aplicado al código fuente de un programa. Cada vez que un desarrollador realiza una modificación y la registra en el sistema, este guarda una fotografía del estado del proyecto en ese momento exacto, junto con información sobre quién realizó el cambio y con qué propósito. Este conjunto de fotografías ordenadas cronológicamente constituye el historial del proyecto, que puede consultarse, compararse y restaurarse en cualquier momento.

Sommerville (2016) destaca que el control de versiones es uno de los pilares del aseguramiento de la calidad en ingeniería de software, ya que proporciona la trazabilidad necesaria para entender la evolución de un sistema, identificar cuándo y por qué se introdujo un error, y revertir cambios problemáticos sin afectar el trabajo realizado posteriormente.

Los conceptos fundamentales que componen un sistema de control de versiones son:

- Repositorio: almacén central donde se guarda el historial completo del proyecto con todas sus versiones.
- Commit o confirmación: acción de registrar un conjunto de cambios en el repositorio con un mensaje descriptivo.

- Rama o branch: línea de desarrollo independiente que permite trabajar en una funcionalidad sin afectar el código principal.
- Fusión o merge: proceso de integrar los cambios de una rama con otra, unificando el trabajo paralelo.
- Conflicto: situación que ocurre cuando dos desarrolladores modifican la misma parte de un archivo de manera incompatible.
- Etiqueta o tag: marcador que identifica un punto específico en la historia del proyecto, generalmente una versión lanzada.
- Clon: copia completa de un repositorio, incluido su historial, en el equipo de un desarrollador.

3.2 Tipos de Sistemas de Control de Versiones

A lo largo de la historia del desarrollo de software, han surgido diferentes enfoques para implementar el control de versiones. Cada tipo responde a necesidades particulares y presenta características, ventajas y limitaciones distintas. Los cuatro grandes modelos son:

3.2.1 Sistemas Locales

Los sistemas de control de versiones locales son la forma más básica de versionamiento. Funcionan exclusivamente en el equipo del desarrollador, sin involucrar ningún servidor externo. Un ejemplo histórico representativo es RCS (Revision Control System), desarrollado en 1982 por Walter Tichy, que almacenaba los cambios entre versiones en formato de parches binarios dentro del mismo sistema de archivos local.

Aunque estos sistemas resolvieron el problema más básico de guardar múltiples versiones de un archivo, presentaron una limitación crítica: la imposibilidad de colaborar con otros desarrolladores. Si dos personas necesitaban trabajar en el mismo proyecto, debían intercambiar archivos manualmente, lo cual generaba confusión, pérdida de cambios y errores difíciles de rastrear. En la práctica, estos sistemas son equivalentes a guardar copias del

proyecto con nombres como proyecto_v1, proyecto_final, proyecto_final2, una práctica que aún hoy en día es común entre desarrolladores sin formación en control de versiones.

3.2.2 Sistemas Centralizados (CVCS)

Los sistemas de control de versiones centralizados surgieron para resolver el problema de la colaboración. En este modelo, existe un único servidor central que contiene el repositorio con toda la historia del proyecto. Los desarrolladores realizan una descarga (checkout) del código al que necesitan acceder, trabajan localmente en sus copias y luego envían (commit) sus cambios al servidor central.

Ejemplos representativos de sistemas centralizados son CVS (Concurrent Versions System), desarrollado en 1986, y su sucesor Subversion (SVN), lanzado en el año 2000 por CollabNet. Pressman y Maxim (2021) señalan que durante la primera década del siglo XXI, SVN fue el estándar de facto en la industria del software, siendo ampliamente adoptado por empresas como Apache Foundation y Google.

El modelo centralizado ofrece ventajas como la simplicidad conceptual, el control granular sobre permisos de acceso por directorio y la facilidad para auditar quién tiene acceso a qué parte del código. Sin embargo, su debilidad estructural más importante es el servidor único como punto de falla: si el servidor central falla o se daña sin un respaldo reciente, el equipo pierde el historial completo del proyecto. Adicionalmente, la mayoría de operaciones requieren conexión a la red, lo que hace que el trabajo sea lento o imposible cuando no hay conectividad.

3.2.3 Sistemas Distribuidos (DVCS)

Los sistemas de control de versiones distribuidos representan el paradigma dominante en el desarrollo de software moderno. En este modelo, cada desarrollador posee una copia completa del repositorio en su equipo local, incluyendo toda la historia del proyecto. Esto significa que la gran mayoría de las operaciones (ver historial, crear ramas, hacer commits,

comparar versiones) son instantáneas porque se realizan localmente, sin necesidad de comunicarse con ningún servidor.

Git, creado en 2005 por Linus Torvalds para gestionar el desarrollo del kernel de Linux, es el sistema distribuido más utilizado en el mundo. Mercurial, también lanzado en 2005, fue otro sistema distribuido popular, especialmente en proyectos de Mozilla y Python, aunque en la actualidad ha cedido terreno frente a Git. Bazaar fue otra alternativa distribuida promovida por Canonical para el desarrollo de Ubuntu.

La arquitectura distribuida ofrece múltiples beneficios: el trabajo offline es completamente funcional, la velocidad de las operaciones locales es superior, no existe un punto único de falla ya que cada clon es un respaldo completo, y el modelo de ramas es significativamente más potente y flexible que en los sistemas centralizados. La popularidad de Git se refleja en los datos de la encuesta anual de Stack Overflow (2023), donde el 93.87% de los desarrolladores encuestados declaró utilizar Git como su herramienta principal de control de versiones.

3.2.4 Sistemas Basados en Bloqueos (Pessimistic Locking)

El modelo de bloqueo pesimista, también denominado checkout exclusivo, es una estrategia de concurrencia en la que un desarrollador debe solicitar explícitamente el permiso para editar un archivo antes de poder modificarlo. El sistema otorga un bloqueo sobre ese archivo, impidiendo que otros miembros del equipo lo editen simultáneamente. Una vez que el desarrollador termina sus cambios y los confirma, el bloqueo se libera y otros pueden editarlo.

Microsoft Visual SourceSafe fue uno de los sistemas más conocidos que implementaba este modelo. Team Foundation Version Control (TFVC), de Microsoft, también ofrece esta modalidad. El bloqueo pesimista es especialmente útil en proyectos que manejan archivos binarios que no pueden fusionarse automáticamente, como archivos de diseño gráfico, modelos 3D, documentos Word o archivos de bases de datos. En estos casos, permitir que dos personas editen el mismo archivo simultáneamente generaría conflictos irresolubles automáticamente.

La desventaja principal del bloqueo pesimista en proyectos de código fuente es que puede crear cuellos de botella significativos: si un desarrollador bloquea un archivo crítico y no lo libera oportunamente, por ejemplo porque está de vacaciones, bloquea el trabajo de todo el equipo. Además, este modelo asume que los conflictos ocurrirán con frecuencia, lo cual es una suposición conservadora que no refleja la realidad de la mayoría de proyectos de software.

3.2.5 Sistemas Basados en Fusión Optimista (Optimistic Merging)

El modelo de fusión optimista es el enfoque opuesto al bloqueo pesimista. En lugar de asumir que los conflictos ocurrirán frecuentemente, parte del supuesto de que la mayor parte del tiempo los desarrolladores trabajan en partes diferentes del código y sus cambios pueden fusionarse automáticamente sin intervención humana. Bajo este modelo, múltiples desarrolladores pueden editar el mismo archivo simultáneamente; solo cuando sus cambios entran en conflicto directo (es decir, cuando modificaron exactamente las mismas líneas de código de manera diferente) el sistema solicita que un desarrollador resuelva el conflicto manualmente.

Git, Subversion y la mayoría de los sistemas modernos implementan el modelo optimista por defecto. La práctica ha demostrado que este supuesto es correcto en la gran mayoría de los casos: en proyectos bien organizados con equipos que trabajan en funcionalidades separadas mediante ramas independientes, los conflictos reales son relativamente infrecuentes y generalmente fáciles de resolver cuando se producen.

La siguiente tabla presenta una comparativa entre los modelos de control de versiones descritos:

Característica	Local	Centralizado	Distribuido	Bloqueos
Colaboración	No	Sí	Sí	Sí
Trabajo offline	Sí	Limitado	Sí completo	Limitado
Punto de falla único	No aplica	Sí	No	Sí
Velocidad operaciones	Alta local	Baja (red)	Alta local	Baja (red)
Concurrencia	No aplica	Optimista	Optimista	Bloqueo
Complejidad	Muy baja	Baja	Media	Baja
Ejemplos	RCS	SVN, CVS	Git, Mercurial	VSS, TFVC
Uso actual	Obsoleto	Legado	Dominante	Nicho

4. Ventajas de las Herramientas de Versionamiento

Las herramientas de control de versiones aportan beneficios que trascienden el simple almacenamiento de archivos. Su impacto se extiende a múltiples dimensiones del proceso de desarrollo de software, desde la calidad del producto final hasta la eficiencia del equipo y la visibilidad del estado del proyecto. A continuación se analizan las ventajas más relevantes.

4.1 Calidad del Software

El control de versiones contribuye directamente a la calidad del software mediante varios mecanismos. En primer lugar, proporciona la capacidad de revertir cambios de manera segura: si una nueva funcionalidad introduce un error o degrada el rendimiento, el equipo puede volver a cualquier estado anterior del código con un único comando, sin perder el trabajo realizado después de ese punto. Esta capacidad elimina el miedo a experimentar y refactorizar, dos actividades esenciales para mejorar la calidad del código.

En segundo lugar, las herramientas modernas de versionamiento se integran con sistemas de integración continua (CI) que ejecutan automáticamente las pruebas del software cada vez que se registra un nuevo cambio. Esto garantiza que el código que ingresa al repositorio principal siempre pase un conjunto mínimo de verificaciones de calidad. Herramientas como GitHub Actions permiten configurar flujos de trabajo que ejecutan linters, pruebas unitarias, pruebas de integración y análisis de seguridad automáticamente en cada commit o pull request.

En tercer lugar, el proceso de revisión de código (code review) que facilitan las plataformas de versionamiento como GitHub o GitLab es un mecanismo probado de mejora de calidad. Antes de que los cambios de un desarrollador sean incorporados al código principal, otro miembro del equipo los revisa, detecta posibles errores de lógica, sugiere mejoras y verifica el cumplimiento de los estándares de codificación.

4.2 Visibilidad y Trazabilidad

Una de las ventajas más valiosas del control de versiones es la visibilidad completa sobre la evolución del proyecto. El historial de commits proporciona una bitácora detallada que responde a las preguntas más importantes en cualquier proyecto de software: qué cambió, quién lo cambió, cuándo ocurrió el cambio y por qué se realizó (a través del mensaje del commit). Esta información es invaluable tanto para el equipo de desarrollo como para las partes interesadas en el proyecto.

La trazabilidad que ofrece el control de versiones permite vincular cada cambio en el código con el requerimiento o el error que lo originó. Por ejemplo, en un proyecto como Trueque Match, es posible identificar exactamente qué commit implementó la funcionalidad de autenticación con JWT, qué desarrollador realizó ese cambio y en qué fecha, lo cual facilita enormemente el diagnóstico de problemas en producción.

Las herramientas modernas como GitHub ofrecen además gráficos de contribución, mapas de actividad, estadísticas de commits por desarrollador y visualizaciones del historial de ramas que proporcionan una vista de alto nivel del estado y el progreso del proyecto en tiempo real.

4.3 Aceleración del Desarrollo

Contrario a lo que podría pensarse, la adopción de herramientas de versionamiento no ralentiza el desarrollo, sino que lo acelera de manera significativa. El sistema de ramas permite que múltiples desarrolladores, o incluso un único desarrollador, trabajen en paralelo en diferentes funcionalidades sin que una interfiera con la otra. Mientras una rama implementa la pantalla de chat, otra puede estar corrigiendo un error en el módulo de autenticación, y una tercera puede estar preparando la nueva versión del APK para revisión.

La posibilidad de crear ramas con un costo prácticamente nulo en Git elimina el miedo a explorar nuevas ideas. Un desarrollador puede crear una rama experimental, probar una nueva arquitectura o tecnología, y si no funciona, simplemente descartarla sin afectar el código principal. Si funciona, puede fusionarla cuando esté lista. Esta libertad para experimentar acelera la innovación y reduce el tiempo necesario para tomar decisiones técnicas.

Adicionalmente, las plataformas de hosting como GitHub ofrecen herramientas de gestión de proyectos integradas (issues, milestones, project boards) que, al estar vinculadas directamente con el repositorio de código, crean un ciclo virtuoso donde la planificación y la implementación se retroalimentan continuamente, acelerando el ciclo de entrega de valor.

4.4 Colaboración Efectiva

El control de versiones transforma radicalmente la manera en que los equipos colaboran. Sin estas herramientas, la coordinación entre desarrolladores requiere procesos manuales propensos a errores: enviar archivos por correo electrónico, copiar carpetas con sufijos como `_version2` o `_final`, y resolver manualmente qué cambios de quién deben quedar en la versión definitiva. Con un sistema de control de versiones distribuido como Git, cada desarrollador tiene autonomía completa para avanzar en su trabajo y el sistema se encarga de facilitar la integración.

Los pull requests y merge requests, funcionalidades ofrecidas por las plataformas de alojamiento como GitHub y GitLab, son el mecanismo estándar de colaboración en equipos de desarrollo modernos. Permiten que un desarrollador proponga formalmente la incorporación de sus cambios al código principal, que otros miembros del equipo los revisen, comenten línea por línea, sugieran modificaciones y finalmente los aprueben. Este proceso no solo mejora la calidad del código sino que también facilita la transferencia de conocimiento entre miembros del equipo.

4.5 Respaldo y Recuperación ante Desastres

En un sistema de control de versiones distribuido, cada clon del repositorio es un respaldo completo del proyecto, incluida toda su historia. Esto significa que incluso si el servidor principal falla o es destruido, cualquier desarrollador que tenga una copia local puede restaurar el repositorio completo. Esta característica hace que los proyectos gestionados con

Git sean inherentemente más seguros que los proyectos almacenados en una única ubicación, ya sea local o en la nube.

Las plataformas como GitHub ofrecen adicionalmente copias de seguridad automáticas, distribución geográfica de los datos y garantías de disponibilidad del 99.9% o superior, lo que las convierte en una capa de protección adicional muy valiosa, especialmente para proyectos en desarrollo activo.

4.6 Documentación Implícita del Proyecto

Un repositorio bien gestionado, con mensajes de commit descriptivos y una estrategia de ramas clara, se convierte en la documentación más actualizada y precisa del proyecto. A diferencia de los documentos de diseño o los comentarios en el código, que frecuentemente quedan desactualizados, el historial de commits refleja siempre la realidad del código tal como existe en cada momento. Herramientas como `git log`, `git blame` y `git bisect` permiten navegar esta documentación implícita de manera eficiente para responder preguntas técnicas específicas.

5. Principales Herramientas de Control de Versiones

El mercado ofrece diversas herramientas de control de versiones, tanto sistemas locales como plataformas de alojamiento en la nube. A continuación se presentan las más relevantes en el contexto del desarrollo de software moderno.

5.1 Git

Git es el sistema de control de versiones distribuido más utilizado en el mundo. Fue creado en abril de 2005 por Linus Torvalds con el objetivo específico de gestionar el desarrollo del kernel de Linux, después de que la comunidad del kernel decidiera dejar de usar el sistema propietario BitKeeper. En solo dos semanas, Torvalds escribió la primera versión funcional de Git, diseñada con principios claros: velocidad, diseño simple, soporte robusto para desarrollo no lineal (miles de ramas paralelas), completamente distribuido y capaz de manejar proyectos del tamaño del kernel de Linux de manera eficiente.

Git trabaja con tres áreas conceptuales fundamentales: el directorio de trabajo donde el desarrollador edita los archivos, el área de preparación (staging area o índice) donde se seleccionan los cambios que van a incluirse en el próximo commit, y el repositorio local donde se almacena el historial. Esta separación en tres áreas proporciona un nivel de control muy granular sobre qué cambios quedan registrados y cómo se organiza el historial del proyecto.

5.2 GitHub

GitHub es la plataforma de alojamiento de repositorios Git más popular del mundo. Lanzada en 2008 por Tom Preston-Werner, Chris Wanstrath y PJ Hyett, fue adquirida por Microsoft en 2018 por 7.500 millones de dólares. Con más de 100 millones de usuarios registrados y más de 420 millones de repositorios según datos de la propia plataforma (GitHub, 2024), GitHub se ha consolidado como el centro neurálgico del desarrollo de software colaborativo a nivel mundial.

Más allá de ser un servidor Git, GitHub es una plataforma completa de desarrollo colaborativo que incluye: pull requests para revisión de código, issues para gestión de tareas y errores, GitHub Actions para integración y despliegue continuos, GitHub Pages para publicación de sitios web estáticos, GitHub Packages para distribución de paquetes de software, y GitHub Projects para gestión visual de proyectos con tableros tipo Kanban.

5.3 GitLab

GitLab es una plataforma DevOps completa de código abierto, lanzada en 2011 por Dmitriy Zaporozhets y Sytse Sijbrandij. Su principal diferenciador frente a GitHub es que puede instalarse en servidores propios de manera completamente gratuita (GitLab Community Edition), lo cual lo hace especialmente atractivo para empresas con requisitos estrictos de privacidad, cumplimiento normativo o control total sobre su infraestructura.

GitLab se distingue por ofrecer en una única plataforma todas las etapas del ciclo DevOps: planificación (issues, wikis, tableros), control de versiones (repositorios Git), integración continua (GitLab CI/CD con uno de los sistemas más potentes del mercado), revisión de código (merge requests), registro de contenedores Docker, monitoreo y análisis de seguridad. Esta integración nativa elimina la necesidad de conectar múltiples herramientas de diferentes proveedores.

5.4 Bitbucket

Bitbucket es la plataforma de alojamiento de repositorios Git de Atlassian, la empresa detrás de Jira, Confluence y Trello. Lanzada en 2008, inicialmente como plataforma para repositorios Mercurial, migró su soporte principal a Git en 2011. Su principal ventaja competitiva es la integración nativa y profunda con el ecosistema Atlassian: los commits de Bitbucket pueden vincularse directamente con issues de Jira, la documentación técnica puede gestionarse en Confluence, y los proyectos pueden visualizarse en tableros de Trello.

Bitbucket ofrece repositorios privados ilimitados en su plan gratuito para equipos de hasta cinco personas, lo que lo hace atractivo para equipos pequeños o startups. Bitbucket Pipelines es su sistema de integración y despliegue continuos, integrado directamente en la plataforma.

5.5 Apache Subversion (SVN)

Apache Subversion, comúnmente conocido como SVN, es un sistema de control de versiones centralizado desarrollado originalmente por CollabNet en 2000 como sucesor de CVS, y donado a la Apache Software Foundation en 2009. Aunque ha perdido terreno significativo frente a Git en los últimos años, SVN sigue siendo utilizado en muchas organizaciones con proyectos heredados (legacy) y en contextos donde el modelo centralizado ofrece ventajas específicas, como proyectos con grandes archivos binarios o requisitos de control de acceso muy granulares.

Una característica que distingue a SVN de Git es su manejo de números de revisión globales y secuenciales: cada commit tiene un número de revisión único que incrementa de manera lineal (r1, r2, r3...), lo que facilita referencias inequívocas a versiones específicas del proyecto y simplifica la auditoría en contextos regulados.

5.6 Azure DevOps (TFVC y Azure Repos)

Microsoft Azure DevOps es una plataforma integral de desarrollo que incluye Azure Repos para el control de versiones. Azure Repos admite dos tipos de repositorios: Team Foundation Version Control (TFVC), que es un sistema centralizado con soporte para bloqueos pesimistas, y repositorios Git estándar. Azure DevOps es especialmente popular en entornos empresariales que ya utilizan el ecosistema Microsoft (Azure, Visual Studio, .NET), ya que la integración entre sus componentes es nativa y profunda.

La siguiente tabla presenta una comparativa de las principales plataformas de versionamiento:

Criterio	GitHub	GitLab	Bitbucket	SVN	Azure DevOps
Tipo de VCS	Git (DVCS)	Git (DVCS)	Git (DVCS)	Centralizado	Git / TFVC
Precio base	Gratis	Gratis	Gratis	Gratis (local)	Gratis (básico)
Self-hosted	Enterprise	Sí (gratis)	Datacenter	Sí	Azure Server
CI/CD integrado	Actions	GitLab CI/CD	Pipelines	Externo	Azure Pipelines
Revisión de código	Pull Request	Merge Request	Pull Request	Externo	Pull Request
Gestión de proyectos	Projects	Issues+Boards	Jira integrado	Externo	Boards
Usuarios activos	> 100 M	> 30 M	> 10 M	Legado	Empresarial
Integración ecosistema	Amplia	DevOps nativo	Atlassian	Limitada	Microsoft
Mejor para	Open source, equipos	Empresas privadas	Equipos Jira	Proyectos legacy	Ecosistema MS

6. Cómo las Herramientas de Versionamiento Optimizan el Control de Versiones

Las herramientas modernas de control de versiones no solo resuelven el problema básico de guardar múltiples versiones de un archivo, sino que introducen flujos de trabajo y mecanismos que optimizan todo el proceso de desarrollo de software. A continuación se describen los mecanismos de optimización más relevantes.

6.1 Estrategias de Ramificación (Branching Strategies)

Una de las contribuciones más importantes de las herramientas modernas de versionamiento es la posibilidad de implementar estrategias de ramificación estructuradas que organizan el flujo de trabajo del equipo. Las estrategias más utilizadas en la industria son:

Git Flow:

Propuesto por Vincent Driessen en 2010, Git Flow define un conjunto fijo de tipos de ramas con responsabilidades específicas: `main` para el código en producción, `develop` para integración de nuevas funcionalidades, `feature/*` para el desarrollo de cada nueva característica, `release/*` para la preparación de nuevas versiones, y `hotfix/*` para correcciones urgentes en producción. Es especialmente adecuado para proyectos con ciclos de lanzamiento definidos.

GitHub Flow:

Una estrategia más simple propuesta por GitHub que consiste en mantener siempre una rama `main` estable y crear ramas de corta duración para cada nueva funcionalidad o corrección. Los cambios se proponen mediante pull requests, se revisan y se fusionan a `main`. Es ideal para proyectos con despliegue continuo donde se publican múltiples versiones por día.

Trunk-Based Development:

Este enfoque promueve que todos los desarrolladores integren sus cambios directamente a la rama principal (trunk o main) al menos una vez al día, usando ramas de muy corta duración o ninguna. Requiere una cultura sólida de pruebas automatizadas y feature flags para que las funcionalidades incompletas no afecten a los usuarios.

6.2 Integración Continua y Despliegue Continuo (CI/CD)

La optimización más transformadora que ofrecen las herramientas modernas de versionamiento es la integración con pipelines de CI/CD. GitHub Actions, GitLab CI/CD y Bitbucket Pipelines permiten definir flujos de trabajo automatizados que se ejecutan en respuesta a eventos del repositorio (push, pull request, creación de tag, entre otros).

Para el proyecto Trueque Match, un pipeline de CI/CD configurado en GitHub Actions podría ejecutar automáticamente las siguientes etapas cada vez que se hace un push a la rama develop: verificación del linter ESLint para detectar errores de código, ejecución de pruebas unitarias, compilación del bundle de React Native y generación del APK mediante Expo EAS Build. Este nivel de automatización garantiza que errores que podrían llegar a producción sean detectados en cuestión de minutos después de ser introducidos.

6.3 Revisión de Código Estructurada

Las plataformas de versionamiento han estandarizado el proceso de revisión de código mediante los pull requests o merge requests. Este mecanismo crea un espacio formal donde los cambios propuestos son discutidos, inspeccionados y aprobados antes de ser incorporados al código principal. GitHub permite configurar reglas de protección de ramas que hacen obligatoria la aprobación de al menos un revisor antes de permitir la fusión, garantizando que ningún cambio ingrese al código principal sin revisión.

6.4 Etiquetado Semántico de Versiones

Las herramientas de versionamiento permiten etiquetar (tag) commits específicos para marcar versiones oficiales del software. La convención más ampliamente adoptada es el versionamiento semántico (SemVer), que define el formato MAYOR.MENOR.PARCHE, donde MAYOR indica cambios incompatibles con versiones anteriores, MENOR indica nuevas funcionalidades compatibles con versiones anteriores, y PARCHE indica correcciones de errores compatibles con versiones anteriores.

En el contexto de Trueque Match, la primera versión funcional con las seis pantallas completadas y el APK generado podría etiquetarse como v1.0.0. Una corrección de un error en el módulo de chat sería v1.0.1, y la adición de la funcionalidad de favoritos sería v1.1.0.

6.5 Resolución Asistida de Conflictos

Las herramientas modernas de versionamiento han desarrollado algoritmos de fusión cada vez más inteligentes que pueden resolver automáticamente la gran mayoría de conflictos que surgen cuando dos desarrolladores modifican el mismo archivo. Los algoritmos de merge de tres vías utilizados por Git comparan la versión común (ancestro), los cambios del desarrollador A y los cambios del desarrollador B, y fusionan automáticamente los cambios que no interfieren entre sí. Solo cuando los cambios son realmente incompatibles, es decir, cuando ambos modificaron exactamente la misma línea de código de manera diferente, el sistema solicita la intervención humana.

7. Selección de Herramientas para Trueque Match

7.1 Características del Proyecto

Trueque Match es una aplicación móvil y web desarrollada con React Native y Expo para el cliente Android, React.js para el cliente web y Node.js con Express para el backend. El proyecto se desarrolla de manera individual en el marco del programa SENA, utilizando Windows como sistema operativo, Visual Studio Code como entorno de desarrollo, MariaDB como sistema de gestión de bases de datos y el dispositivo Redmi 9 para pruebas del APK.

Las características específicas del proyecto que deben considerarse al seleccionar las herramientas de versionamiento son: desarrollo individual con posibilidad de trabajo colaborativo futuro, presupuesto nulo para licencias de software, stack tecnológico completamente basado en JavaScript y TypeScript, integración necesaria con Expo EAS para la generación del APK y necesidad de evidenciar el proceso de desarrollo para los instructores del SENA.

7.2 Criterios de Selección

- Gratuidad: el proyecto no cuenta con presupuesto para herramientas de pago.
- Compatibilidad: debe funcionar perfectamente en Windows y con el ecosistema JavaScript/TypeScript/React Native.
- Integración con Expo: la cuenta gijo del estudiante en Expo EAS ya está vinculada, lo que favorece plataformas con las que Expo tiene integración nativa.
- Adopción en la industria: priorizar herramientas que sean demandadas en el mercado laboral colombiano e internacional.
- Facilidad de aprendizaje: el estudiante está en proceso de formación y necesita herramientas con documentación amplia y comunidad activa.
- Evidenciabilidad: la herramienta debe permitir mostrar el historial de desarrollo de manera clara para los procesos evaluativos del SENA.

7.3 Herramienta Seleccionada: Git + GitHub

Con base en el análisis realizado a lo largo de este informe y los criterios definidos para el proyecto Trueque Match, la selección de herramientas de versionamiento es Git como sistema de control de versiones distribuido local y GitHub como plataforma de alojamiento y colaboración en la nube.

7.3.1 Justificación de la Selección de Git

Git es la herramienta de control de versiones correcta para Trueque Match por las siguientes razones fundamentales:

- Es completamente gratuito y de código abierto, sin ninguna restricción de uso en proyectos académicos o comerciales.
- Es el estándar absoluto de la industria con una adopción del 93.87% según Stack Overflow (2023), lo que garantiza que las competencias adquiridas son directamente transferibles al mercado laboral.
- Su naturaleza distribuida permite trabajar con funcionalidad completa incluso sin conexión a internet, lo cual es relevante dado el contexto de trabajo desde casa.
- Se integra nativamente con Visual Studio Code mediante la extensión oficial de Git, permitiendo realizar commits, ver diferencias y gestionar ramas directamente desde el editor.
- El modelo de ramas de Git es el más flexible y potente disponible, permitiendo implementar una estrategia GitHub Flow adaptada al desarrollo individual de Trueque Match.

7.3.2 Justificación de la Selección de GitHub

GitHub es la plataforma de alojamiento seleccionada por sobre GitLab y Bitbucket por las siguientes razones específicas al contexto del proyecto:

- La cuenta gijo del estudiante en la plataforma Expo ya está configurada con integración a GitHub, lo que facilita enormemente el proceso de compilación automática del APK mediante Expo EAS Build.
- GitHub ofrece repositorios privados ilimitados en su plan gratuito, permitiendo proteger el código del proyecto durante su desarrollo sin costo alguno.
- GitHub Actions permite automatizar en el futuro la ejecución del linter ESLint y la generación del APK con Expo EAS cada vez que se realice un push, sin configuración de infraestructura adicional.
- El portafolio de proyectos en GitHub es el activo profesional más reconocido en el mercado laboral de desarrollo de software. Tener el proyecto Trueque Match en GitHub con un historial de commits bien documentado es una ventaja directa para el perfil profesional del estudiante.
- GitHub tiene la comunidad más grande, la documentación más extensa y el mayor ecosistema de integraciones de todas las plataformas analizadas, lo que facilita encontrar soluciones a problemas específicos durante el desarrollo.

7.3.3 Estrategia de Versionamiento para Trueque Match

Se adopta una versión simplificada de GitHub Flow adaptada al desarrollo individual del proyecto. La estructura de ramas propuesta es:

- Rama main: contiene siempre el código estable, revisado y funcional. El APK oficial se genera desde esta rama.
- Rama develop: rama de integración donde se consolidan las nuevas funcionalidades antes de pasar a main.
- Ramas feature/*: una rama por cada nueva pantalla, módulo o funcionalidad. Ejemplos: feature/pantalla-login, feature/modelo-bd, feature/api-autenticacion.
- Ramas fix/*: para correcciones de errores específicos. Ejemplos: fix/validacion-correo, fix/badge-chat.

La convención de mensajes de commit adoptada será Conventional Commits, que define el formato: tipo(alcance): descripción breve. Los tipos son feat para nuevas funcionalidades, fix para correcciones, docs para documentación, style para cambios de formato, refactor para reestructuración de código, test para pruebas y chore para tareas de mantenimiento.

La siguiente tabla resume la selección final de herramientas y su justificación:

Herramienta	Rol	Justificación Principal
Git 2.x	VCS distribuido local	Estándar industria, gratuito, trabajo offline completo
GitHub (plan Free)	Repositorio remoto + CI/CD	Integración Expo EAS, portafolio profesional, gratuito
GitHub Actions	Automatización CI/CD	Linter + build APK automático en cada push
GitHub Flow	Estrategia de ramas	Simple, adecuada para desarrollo individual/pequeño equipo
Conventional Commits	Estándar de mensajes	Historial legible, documentación implícita del desarrollo

8. Conclusiones

El control de versiones es una práctica no negociable en cualquier proyecto de software profesional, independientemente de su tamaño o del número de desarrolladores involucrados. A través de esta investigación se ha evidenciado que los sistemas modernos de control de versiones, especialmente los distribuidos como Git, ofrecen un conjunto de ventajas que impactan positivamente en todas las dimensiones del desarrollo de software: calidad, velocidad, visibilidad, colaboración y resiliencia ante fallos.

La evolución desde los sistemas locales hasta los sistemas distribuidos refleja la creciente complejidad y el carácter colaborativo del desarrollo de software actual. El modelo distribuido representado por Git ha demostrado ser el más adecuado para la realidad del desarrollo moderno, donde los equipos son frecuentemente distribuidos geográficamente y la velocidad de entrega es una ventaja competitiva crítica.

Para el proyecto Trueque Match, la combinación de Git como sistema de control de versiones y GitHub como plataforma de alojamiento constituye la decisión técnica más fundamentada, considerando el contexto académico, el stack tecnológico, la integración con Expo EAS y los objetivos de formación profesional del estudiante. La implementación de GitHub Flow como estrategia de ramas y Conventional Commits como estándar de mensajes complementa esta selección con una estructura de trabajo clara y profesional.

Más allá del beneficio inmediato para el desarrollo de Trueque Match, el dominio de estas herramientas representa una inversión directa en el perfil profesional del estudiante, dado que Git y GitHub son competencias fundamentales y ampliamente demandadas en el mercado laboral del desarrollo de software en Colombia y a nivel internacional.

9. Referencias

- Atlassian. (2024). Git tutorials and training. Atlassian. <https://www.atlassian.com/git>
- Chacon, S., y Straub, B. (2014). Pro Git (2.a ed.). Apress. <https://git-scm.com/book/en/v2>
- Collins-Sussman, B., Fitzpatrick, B. W., y Pilato, C. M. (2011). Version Control with Subversion. O'Reilly Media. <https://svnbook.red-bean.com>
- Conventional Commits. (2024). A specification for adding human and machine readable meaning to commit messages. <https://www.conventionalcommits.org>
- Driessen, V. (2010). A successful Git branching model. nvie.com. <https://nvie.com/posts/a-successful-git-branching-model/>
- GitHub. (2024). About GitHub. GitHub Inc. <https://github.com/about>
- GitLab. (2024). The complete DevSecOps platform. GitLab Inc. <https://about.gitlab.com>
- Loeliger, J., y McCullough, M. (2012). Version Control with Git (2.a ed.). O'Reilly Media.
- Pressman, R. S., y Maxim, B. R. (2021). Ingeniería del software: Un enfoque práctico (8.a ed.). McGraw-Hill Education.
- Semantic Versioning. (2023). Semantic Versioning 2.0.0. <https://semver.org>
- Sommerville, I. (2016). Ingeniería de software (10.a ed.). Pearson Educación.
- Stack Overflow. (2023). Stack Overflow Developer Survey 2023. Stack Overflow. <https://survey.stackoverflow.co/2023>

Tichy, W. F. (1985). RCS: A system for version control. *Software: Practice and Experience*, 15(7), 637-654. <https://doi.org/10.1002/spe.4380150703>